

rwMotorTorque

1 Executive Summary

This module maps a desired torque to control the spacecraft, and maps it to the available reaction wheels using a minimum norm inverse fit. All numeric computation uses single-precision (`float` / `fp32`).

The optional wheel availability message is used to include or exclude particular reaction wheels from the torque solution. The desired control torque can be mapped onto particular orthogonal control axes to implement a partial solution for the overall attitude control torque.

The module also provides an optional reaction-wheel null-space despin term that drives the wheel speeds toward a desired set without producing any net body torque. This is enabled by a non-zero `omegaGain` together with the optional RW speed messages; with the default zero gain (or no speed message linked) the module reduces to the pure control-torque mapping.

The module follows the standard three-layer pattern: the pure `RwMotorTorqueAlgorithm` (Eigen-typed, no messaging), the `RwMotorTorque` Xmera adapter (owns message I/O and payload conversion), and a C-linkage shim (`rwMotorTorqueAlgorithm.c.*`) exposing the algorithm to Ada via FFI. The control axes mapping matrix is supplied as a two-phase configuration property and validated when `reset()` builds the algorithm configuration.

2 Message Connection Descriptions

The following table lists all the module input and output messages. The module msg connection is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
<code>rwMotorTorqueOutMsg</code>	<code>RwMotorTorqueMsgPayload</code>	RW motor torque output message
<code>vehControlInMsg</code>	<code>CmdTorqueBodyMsgPayload</code>	commanded vehicle control torque input message
<code>vehControlIn2Msg</code>	<code>CmdTorqueBodyMsgPayload</code>	(optional) additional commanded vehicle control torque input message
<code>rwParamsInMsg</code>	<code>RWArrayConfigMsgPayload</code>	RW array configuration input message
<code>rwAvailInMsg</code>	<code>RWAvailabilityMsgPayload</code>	(optional) RW device availability message
<code>rwSpeedsInMsg</code>	<code>RWSpeedMsgPayload</code>	(optional) current RW speeds, used by the null-space despin term
<code>rwDesiredSpeedsInMsg</code>	<code>RWSpeedMsgPayload</code>	(optional) desired RW speeds for the null-space despin term (defaults to zero)

3 Module Parameters

The following table lists all the module parameters than can be set. The parameters are optional unless indicated (if not specified default is used).

Table 2: Module Parameters

Parameter Name	Type	Units	Default	Description	Bounds
controlAxes_B (required)	Eigen::Matrix3f	[-]	[0] _{3x3}	array of the control unit axes (axis in each row)	N/A
omegaGain	float	[-]	0	RW null-space despin feedback gain; 0 disables the despin term	≥ 0

4 Model Assumptions and Limitations

This code makes the following assumptions:

- The control mapping matrix $[CB][G_s]$ (constructed using only the available wheels) must be well conditioned: every control axis must be reachable by the available wheels, and the smallest-to-largest singular value ratio must exceed 10^{-2} (condition number below 100). A rank-deficient or ill-conditioned mapping would amplify the command and single-precision round-off, so such a configuration is rejected with an invalid argument exception when the configuration is created.
- It is assumed that the availability of the reaction wheels is known at the time of reset, and does not change after that.
- The control axes and RW spin axes are expected to be unit vectors. They are validated when the configuration is created (a non-unit spin axis is rejected) and canonicalized before use: the RW spin axes are normalized and the control axes are orthonormalized (Gram-Schmidt).
- The null-space despin term requires more than three *available* reaction wheels spanning 3-D for a non-trivial null space. With three (or fewer) available spanning wheels the null space is empty, $[\tau]$ is zero, and the despin term has no effect. A zero **omegaGain** (the default) likewise disables the despin term. The available-wheel geometry $[G_s]$ must also be well conditioned (condition number below 100); a near-coplanar array of more than three wheels is rejected when the configuration is created.

5 Initialization

The module is configured by:

```
module = rwMotorTorque.RwMotorTorque()
module.modelTag = "rwMotorTorque"
control_axes_B = [[1, 0, 0], [0, 1, 0], [0, 0, 0]]
module.controlAxes_B = control_axes_B
```

6 Detailed Model Description

6.1 Module Input and Output Behavior

This module takes an attitude control torque ${}^B\mathbf{L}_r$ and maps the vector onto the specific control axes, $\hat{\mathbf{c}}_i$. This allows only a subset of ${}^B\mathbf{L}_r$ to be implemented with the Reaction Wheels (RWs). The next step is to map reduced control torque onto the available RW spin axes, $\hat{\mathbf{g}}_{s_j}$. The module accounts for the availability of the reaction wheels in the case that not all wheels are functioning appropriately or are undergoing independent analysis.

Assume in this documentation that the number of available RWs is m , while the number of desired control axes $\hat{\mathbf{c}}_i$ is n . The number of installed RWs is N .

The commanded torque message is a required input message and is read in every time step. The RW configuration message is also required, but only read in during reset. If the RW spin axes $\hat{\mathbf{b}}_{s_i}$ change then `reset()` must be called again. The RW availability message is optional. If the availability message is not used, then all installed RWs are assumed to be available. The output message is always the array of RW motor torques.

6.2 Torque Mapping

The `rwMotorTorque` module receives a desired attitude control torque in the body frame ${}^B\mathbf{L}_r$. If two control torques are provided via the second optional control input message, these two are added together to compute ${}^B\mathbf{L}_r$. This torque is the net control torque that should be applied to the spacecraft. Let $\hat{\mathbf{g}}_{s_i}$ be the individual RW spin axis, while $\mathbf{u}_s$ is the m -dimensional array of motor torques. The $3 \times m$ projection matrix $[G_s]$ then maps the control torque on motor torques using

$$[G_s]\mathbf{u}_s = (-{}^B\mathbf{L}_r)$$

The projection matrix is defined as

$$[G_s] = [\hat{\mathbf{g}}_{s_1} \quad \cdots \quad \hat{\mathbf{g}}_{s_m}]$$

Note that here $\mathbf{u}_s$ is the array of available motor torques. The installed set of RWs could be larger than m .

The projection matrix to map a vector in the body frame $\mathcal{B}$ onto the set of control axes $\hat{\mathbf{c}}_i$ is given by

$$[CB] = \begin{bmatrix} {}^B\hat{\mathbf{c}}_1^T \\ \vdots \\ {}^B\hat{\mathbf{c}}_n^T \end{bmatrix}$$

where $n \leq 3$.

To map the requested torque onto the control axes, the first equation is pre-multiplied by $[CB]$ to yield

$$[CB][G_s]\mathbf{u}_s = [CG_s]\mathbf{u}_s = [CB](-{}^B\mathbf{L}_r) = {}^C\mathbf{L}_r$$

Note that $[CG_s]$ is a $n \times m$ matrix.

The module assumes that $m \geq n$ such that there are enough RWs available to implement ${}^C\mathbf{L}_r$. If not, then the output motor torques are set to zero with $\mathbf{u}_s = \mathbf{0}$.

To invert the equation above, the minimum-norm solution

$$\mathbf{u}_s = [CG]^T ([CG][CG]^T)^{-1} {}^C\mathbf{L}_r$$

is used. It is evaluated through the Moore-Penrose pseudo-inverse of $[CG_s]$ computed from a singular value decomposition (a truncated SVD, $[V][\Sigma]^{-1}[U]^T$, with singular values below a relative tolerance treated as zero) rather than by forming and inverting the normal-equation Gram matrix $[CG_s][CG_s]^T$. The SVD form avoids squaring the condition number and is the same construction used by `forceTorqueThrForceMapping`. The controllability and conditioning of $[CG_s]$ are checked from this same decomposition (see Model Assumptions).

The final step is to map the array of available motor torques $\mathbf{u}_s$ onto the output array of installed RW motor torques.

6.3 RW Availability

If the input message `rwAvailInMsg` is linked, then the RW availability message is read in. The torque mapping is only used for RW's whose availability setting is `AVAILABLE`. If it is `UNAVAILABLE` then that RW output torque is set to zero.

6.4 RW Null-Space Despin

On top of the control-torque solution, the module can add a null-space despin torque that drives the RW speeds toward a desired set without disturbing the spacecraft attitude. This capability was previously provided by a separate `rwNullSpace` module and is now part of `rwMotorTorque`.

Let N be the number of installed RWs and $\hat{\mathbf{g}}_{s_i}$ the i^{th} RW spin axis. The $3 \times N$ spin-axis matrix $[G_s]$ holds the **available** wheels' spin axes in their installed columns and zeros for unavailable wheels:

$$[G_s] = [\hat{\mathbf{g}}_{s_1} \cdots \hat{\mathbf{g}}_{s_N}]$$

The null-space projection matrix $[\tau]$ is the orthogonal projector onto the null space of $[G_s]$,

$$[\tau] = [I_{N \times N}] - [V_r][V_r]^T$$

where the columns of $[V_r]$ are the right singular vectors of $[G_s]$ with non-zero singular values (a basis for its row space), obtained from a singular value decomposition. Building $[\tau]$ from orthonormal singular vectors makes $[G_s][\tau] = [0]$ hold to machine precision regardless of conditioning, which avoids the single-precision torque leakage incurred by the algebraically equivalent normal-equation form $[I] - [G_s]^T([G_s][G_s]^T)^{-1}[G_s]$. By construction any torque produced by $[\tau]$ lies in the RW null space and exerts no net torque on the spacecraft, leaving the attitude control solution unaffected.

Let Ω_i be the RW spin speeds and $\Omega_{i,d}$ the desired speeds, with tracking error $\Delta\Omega_i = \Omega_i - \Omega_{i,d}$. The desired despin torque array is

$$\mathbf{d} = -K \Delta\Omega$$

with feedback gain $K = \text{omegaGain} \geq 0$. Mapping it through $[\tau]$ and adding it to the control solution $\mathbf{u}_{s,\text{cont}}$ gives the module output

$$\mathbf{u}_s = \mathbf{u}_{s,\text{cont}} + [\tau] \mathbf{d}$$

The projection matrix $[\tau]$ is precomputed at reset, and its rows for unavailable wheels are zeroed so the despin term never commands torque to a wheel the control mapping excluded. A null space exists only when more than three wheels are available and span 3-D; with three (or fewer) available wheels $[\tau]$ is zero and the despin term vanishes. When more than three wheels are available their geometry must be well conditioned (condition number below 100); a near-coplanar array is rejected when the configuration is created rather than producing an unreliable despin. The current and desired RW speeds are read each update from the optional `rwSpeedsInMsg` and `rwDesiredSpeedsInMsg` (unlinked speeds default to zero).

This availability handling is a deliberate departure from the legacy `rwNullSpace` module, which had no concept of RW availability and built $[\tau]$ from all configured wheels. For configurations with all wheels available the two are identical.

7 Model Functions

The code performs the following functions:

- **Maps control torque vector onto available reaction wheels:** Takes a desired body-frame torque from `CmdTorqueBodyIntMsg` and maps it onto the RW axes.
- **Removes torque from unavailable reaction wheels:** The module observes the availability of the RWs and maps the torques to only available reaction wheels.
- **Adds an optional RW null-space despin torque:** When `omegaGain` is non-zero, the module superimposes a null-space torque that drives the RW speeds toward their desired values without producing any net body torque.